FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Departamento de Ciencias de la Computación.
	Carrera: ITIN Asignatura: Metod. Desarrollo de Software NRC: 30692
	Apellidos y nombres del estudiante: Villacis Arrieta Henry Paul

ANTECEDENTES.

De acuerdo con el libro *Fundamentos de Ingeniería de Software: un enfoque práctico*, específicamente en los capítulos 1 y 2, el presente trabajo aborda los conceptos básicos relacionados con el origen, evolución y características de la ingeniería de software, así como los aspectos fundamentales del producto software y su calidad.

Históricamente, el desarrollo de software se realizaba de forma empírica y sin métodos formales, lo que originó la denominada crisis del software debido a problemas de complejidad, tiempo y costos. Como respuesta a esta problemática, surgió la ingeniería de software como una disciplina que aplica procesos, métodos y herramientas para desarrollar software de manera organizada y eficiente, garantizando atributos de calidad como confiabilidad, mantenibilidad y seguridad (Pressman, 2005; Sommerville, 2011).

En este contexto, comprender los fundamentos de la ingeniería de software permite analizar cómo se desarrollan los sistemas informáticos y la importancia de aplicar metodologías adecuadas para asegurar productos de calidad.

OBJETIVO.

OBJETIVO GENERAL.

Analizar los fundamentos teóricos, metodológicos y de calidad de la ingeniería de software para comprender su impacto en el diseño, desarrollo y mantenimiento eficiente de sistemas informáticos adaptados a las necesidades del mundo real.

OBJETIVOS ESPECÍFICOS.

- Diferenciar la ingeniería de software de otras disciplinas computacionales, destacando su enfoque sistemático y la estructuración de su ciclo de vida frente a la simple codificación.
- Examinar el impacto de las metodologías de desarrollo (ágiles, pesadas e híbridas) y el uso de herramientas de Ingeniería de Software Asistida por Computador (CASE) en la mitigación de la deuda técnica y la optimización de costos.
- Evaluar la importancia de los estándares internacionales, específicamente la norma ISO/IEC 25010, para garantizar que los productos de software cumplan con atributos críticos como la eficiencia de desempeño, mantenibilidad y confiabilidad.

DESARROLLO.

Tarea N°1

1. ¿Qué es la ingeniería de software?

La ingeniería de software es la disciplina que aplica métodos y procesos sistemáticos para el diseño, desarrollo, operación y mantenimiento de sistemas informáticos, con el objetivo de producir software confiable, eficiente y de calidad. Esta disciplina abarca todo el ciclo de vida del software, desde la definición de requisitos hasta su desarrollo, pruebas e implementación,

buscando crear soluciones que respondan a necesidades reales y que puedan mantenerse y mejorar con el tiempo.

2. ¿Cuáles son las actividades fundamentales de la ingeniería de software?

Las actividades fundamentales de la ingeniería de software incluyen el levantamiento de requisitos, donde se identifican las necesidades funcionales y no funcionales del cliente; el diseño y modelado, que consiste en crear la arquitectura o estructura del sistema antes de su desarrollo; la construcción, que implica la escritura del código fuente utilizando distintos lenguajes de programación; la verificación, mediante la cual se realizan pruebas para detectar errores y asegurar el correcto funcionamiento del software; y finalmente el mantenimiento y operación, que permite mantener el sistema funcionando correctamente y adaptarlo a nuevas necesidades a lo largo del tiempo.

3. ¿Cuál es la diferencia entre ingeniería de software y ciencias de la computación?

Ingeniería de Software (SE): Se enfoca en la creación de software sistemático y robusto que satisfaga las necesidades del mundo real, cubriendo aplicaciones a gran escala.

Ciencias de la Computación (CS): Se centra en los fundamentos teóricos y algorítmicos; aquí el software se utiliza como un "lenguaje" para expresar ideas y resolver problemas computacionales abstractos, como visión por ordenador o robótica.

4. ¿Qué es software?

Es el conjunto de programas o instrucciones, procedimientos, datos relacionados y la documentación asociada requerida para desarrollar, operar y mantener un sistema informático. Se diferencia del hardware en que se desarrolla o modifica mediante el intelecto y no se "fabrica" en el sentido industrial clásico.

5. ¿Cuáles son los atributos del buen software?

Un software de alta calidad debe poseer las siguientes características:

- Mantenibilidad: Debe ser fácil de modificar para adaptarse a cambios.
- Confiabilidad (Fiabilidad): Debe ser seguro y generar confianza en su funcionamiento.
- Eficiencia: No debe malgastar los recursos del sistema y debe trabajar de modo rentable.
- Usabilidad: Debe ser fácil de utilizar para el usuario final.
- Funcionalidad: Debe cumplir con los requisitos específicos y expectativas para los cuales fue creado.

6. ¿Cuáles son los principales retos que enfrenta la ingeniería de software?

Los principales retos de la ingeniería de software incluyen gestionar la complejidad de los sistemas, desarrollar software de calidad bajo restricciones de tiempo y presupuesto, garantizar la seguridad de la información e integrar correctamente los requisitos no funcionales como rendimiento y escalabilidad. Además, busca reducir la denominada crisis del software, mejorando la eficiencia en el desarrollo de sistemas complejos.

7. ¿Cuáles son los costos de la ingeniería de software?

Los costos de la ingeniería de software están relacionados con el desarrollo, operación y mantenimiento del software. Estos incluyen la inversión necesaria para construir productos de calidad dentro de límites de tiempo y presupuesto, así como los costos asociados a corregir errores o fallas que puedan surgir durante el desarrollo. Además, una parte importante del costo se encuentra en el mantenimiento y actualización del software después de su implementación, ya que los sistemas deben adaptarse a nuevas necesidades y cambios tecnológicos.

8. ¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?

Métodos Ágiles: Son ideales para reducir la complejidad y ser adaptables.

Métodos Híbridos: Buscan un equilibrio; combinan la flexibilidad de lo ágil con la formalidad y documentación de los métodos más estrictos.

Enfoque actual: En la última década, se ha dado mucha importancia a integrar la ingeniería de software con la de sistemas, priorizando la seguridad y los requisitos no funcionales

9. ¿Qué entiende por paradigmas de la ingeniería de software?

Se entiende como la evolución de los distintos enfoques y modelos de trabajo que han guiado el desarrollo de sistemas a través de la historia. Originalmente, el software se veía como un "arte" sin métodos sistemáticos ni planificación que, con el tiempo, se pasó por el modelo de "codifica y corrige" de los años sesenta hasta llegar a técnicas más estructuradas en los setenta y ochenta, como la programación orientada a objetos. Actualmente, el paradigma se centra en la agilidad y la capacidad de respuesta ante requisitos que cambian constantemente.

10. ¿Cuál es la definición del modelo incremental versus otros paradigmas? Características y diferencias con otras ingenierías.

El modelo incremental se define como un intento de reducir la complejidad frente a los modelos antiguos o "pesados" (como el CMM), permitiendo un desarrollo más adaptable y menos rígido.

Características y diferencias con otras áreas:

- **Ingeniería de Software vs. Ciencias de la Computación:** Mientras que la computación se enfoca en la teoría, los algoritmos y el software como un lenguaje para explorar problemas, la

ingeniería de software es la aplicación práctica para construir sistemas de forma económica y fiable.

- **Ingeniería de Software vs. Ingeniería de Computadoras:** La ingeniería de computadores se centra en el diseño de hardware y dispositivos digitales, mientras que la de software se enfoca en programas que satisfagan necesidades del mundo real.

- **El enfoque del Ingeniero:** A diferencia de otras ramas más teóricas, el ingeniero de software debe resolver problemas complejos bajo restricciones estrictas de tiempo y presupuesto, apoyándose en métodos empíricos para obtener resultados de alta calidad.

Tarea N°2

1. ¿Qué es una metodología de desarrollo de software?

Un conjunto de lenguajes de programación que permiten analizar, diseñar y construir productos software.

Justificación.

Esta respuesta se fundamenta en que el desarrollo de software no es un acto aislado, sino un proceso de ingeniería cuyas actividades están debidamente estandarizadas, secuenciadas y estructuradas dentro de un ciclo de vida. Según Ian Sommerville (2011), un proceso de software es un conjunto de actividades relacionadas que conducen a la elaboración de un producto. Las metodologías integran estos procesos bajo una filosofía (como la agilidad o el enfoque en cascada) para gestionar la complejidad técnica y organizativa.

2. Las actividades estructurales del proceso de desarrollo de software son

Despliegue/comunicación/modelado/diseño.

Justificación.

Roger Pressman gran ingeniero en software propone cinco actividades estructurales básicas que se aplican a cualquier proyecto o proceso de software dependiendo de qué tan grande sea su complejidad. Lo que propone es: comunicación, colaboración con los clientes para entender los objetivos y requisitos. Planeación, la creación de un mapa como es cronograma, riesgos, recursos. Modelado, utilización de modelos para una mejor comprensión del problema como análisis y diseño. Construcción, generar código y pruebas. Despliegue, Entrega y finalización del software al cliente.

3. Una de las razones por la cual se incrementan los costos al agregar funcionalidad.

Práctica de desarrollo deficiente.

Justificación.

Cuando el desarrollo es deficiente como código desordenado, la falta de documentación o arquitectura improvisada. Todo esto genera deuda técnica, al intentar agregar una nueva funcionalidad sobre una base frágil tenemos que los desarrolladores deben parchar el código, se debe volver hacer el código ya que es muy tardado entender un código mal hecho. El resultado de todo esto, lo que debería tomar 5 horas termina siendo más de 20 horas. No se trata solo de escribir código lento, sino de las consecuencias estructurales de hacerlo mal. El costo aumenta porque la ineficiencia acumulada actúa como un "impuesto" sobre cada nueva línea de código que se intenta escribir. Una mala base multiplica el esfuerzo necesario para cualquier cambio futuro.

4. Ingeniería de software asistida por computador significa

Conjunto de herramientas de software integradas para ser utilizadas por otras.

Justificación.

Esta respuesta se basa en el concepto de CASE (*Computer-Aided Software Engineering*). Muchos piensan que la ingeniería en software es una actividad manual, pero requiere de un ecosistema tecnológico que ayuda a la automatización y cohesione las fases de ciclo de vida. El CASE ayuda a crear el software, como son herramientas utilizadas por otras permiten automatizar tareas muy complejas. Como los Upper CASE que son herramientas para planificar, diseñar y analizar. Lower Case son herramientas para generar código, pruebas y el despliegue.

La Ingeniería de Software Asistida por Computador no es solo usar un editor de texto, sino que es la agregación de una infraestructura tecnológica la cual distintas herramientas se ayudan entre sí para estandarizar y agilizar el proceso de desarrollo.

5. La metodología con sus tipos correspondientes de clasificación es:

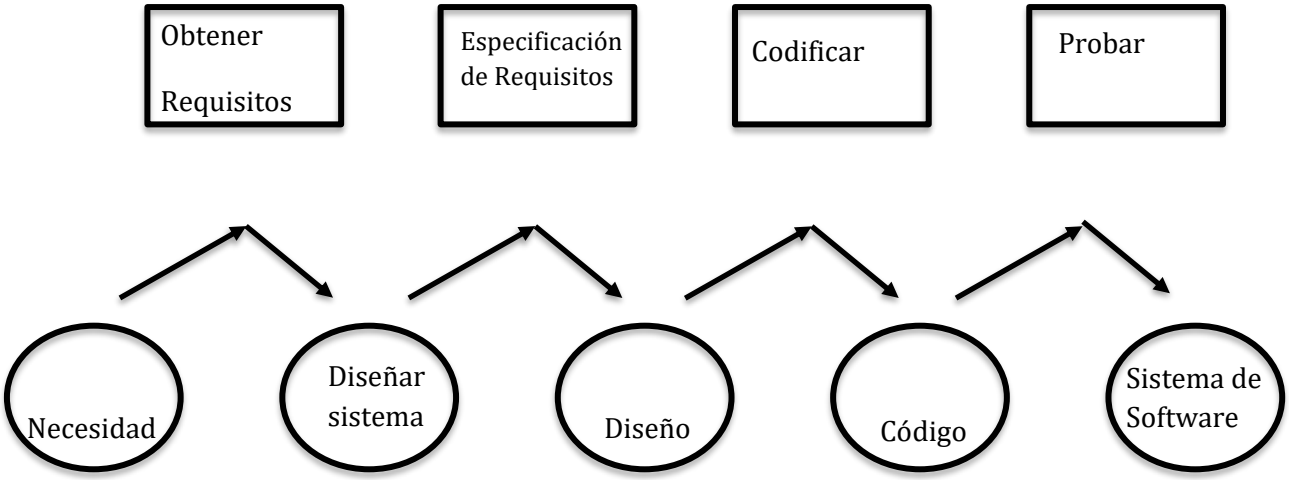
Metodología XP (orientada a objetos-ágil).

Justificación.

Esta clasificación responde a la necesidad de gestionar proyectos donde los requisitos cambian rápidamente, exigiendo una alta capacidad de adaptación y mantenibilidad. El desarrollo bajo metodologías ágiles como XP (Extreme Programming) se apoya frecuentemente en lenguajes orientados a objetos como Java o Python, facilitando la modularidad y la reusabilidad del código.

De acuerdo con Pressman (2005), XP se define como un modelo de proceso ágil que utiliza un enfoque orientado a objetos, enfatizando la participación del cliente y el diseño simple.

6. En el gráfico propuesto, la secuencia ordenada de las figuras es:



Necesidad, obtener requisitos, diseñar sistema, especificación de requisitos, diseño, codificar, código, probar, sistema de software.

Justificación.

Este orden refleja las etapas básicas del proceso de desarrollo de software. Primero se identifica una necesidad, luego se obtienen y especifican los requisitos para definir lo que el sistema debe realizar. Posteriormente se realiza el diseño del sistema, se desarrolla el código mediante la programación y finalmente se prueba el sistema para verificar que funcione correctamente antes de obtener el software final. Este proceso permite desarrollar sistemas de forma organizada y asegurar su correcto funcionamiento.

7. Los modelos de calidad del producto software y los modelos de calidad del proceso de desarrollo de Software son:

ISO/IEC 25010 producto software.

Justificación.

La norma ISO/IEC 25010 forma parte de la serie de estándares ISO/IEC 25000 (SQuaRE) y proporciona un modelo de calidad para productos de software, el cual describe las características y subccaracterísticas que permiten evaluar si un software cumple con los requisitos y expectativas de los usuarios. Estas características incluyen aspectos como funcionalidad, eficiencia, seguridad, confiabilidad, usabilidad y mantenibilidad.

Por esta razón, cuando la pregunta habla de “modelos de calidad del producto software”, la opción correcta es ISO/IEC 25010, ya que esta norma fue creada específicamente para medir y evaluar la calidad de un producto de software mediante un conjunto estructurado de atributos de calidad.

8. La norma ISO/IEC 25010 identifica características y subcaracterísticas de calidad del producto software.

Eficiencia de desempeño es característica.

Justificación.

Esta respuesta se basa en la jerarquía del modelo de calidad. El modelo de calidad de producto de la ISO 25010 se organiza en: características las categorías de alto nivel y subcaracteristicas la cual se basa en los atributos específicos y medibles que componen cada característica. Gracias a las características y subcaracteristicas podemos definir que la Eficiencia de Desempeño está dentro de las 8 categorías de característica, esto habla de que es un pilar importante para evaluar si un software es de calidad.

La eficiencia de desempeño es una característica porque funciona como un recipiente de alto nivel que agrupa atributos relacionado con el rendimiento del software, sin esta característica, el modelo no tiene una forma estandarizada de medir la calidad del software.

9. Según los roles y las responsabilidades correspondientes que opción es correcta.

Verificar los cambios y resolver conflictos corresponde a un desarrollador.

Justificación.

Esta respuesta se basa en el flujo de trabajo técnico y operativo del ciclo de vida del desarrollo de software (SDLC), principalmente dentro de la fase de construcción y el uso de sistemas de control de versiones como puede Git. En metodologías modernas como Agile, DevOps, etc. El desarrollador es propietario de su código desde el inicio hasta que lo integra en la rama principal que se lo puede llamar Main/Master. La resolución de problemas y la verificación son tareas de implementación pura. Como solo el desarrollador tiene el contexto del cambio, es solo el quien tiene el rol capacitado para asegurar que la implementación de su código no afecte a la calidad del producto.

10. Según los objetivos con la correspondiente fase del proceso unificado de desarrollo, que opción es la correcta.

Inicio establece el alcance del proyecto y las condiciones de los límites, criterios de aceptación, lo que se necesita en el producto y lo que no.

Justificación.

Esta respuesta se lo justifica con el proceso unificado de desarrollo (RUP), el cual divide el proyecto en cuatro fases, la fase de inicio que se lo llama Inception, tiene un propósito estratégico

y de definir no de construcción. En la fase de inicio se enfoca en determinar si el proyecto es confiable y que es lo que se va a construir. El alcance, en donde se define la visión del proyecto, si no tenemos un alcance claro el proyecto corre el riesgo del llamado Scope Creep, o el crecimiento descontrolado. Limites, la cual es importante definir que hace y que no hace el sistema, esto evita confusiones con los Stakeholders en fases avanzadas y costosas.

La fase de Inicio es el cimiento del proyecto. No se trata de programar, sino de acotar. Por eso, definir el alcance, los límites y los criterios de aceptación es su razón de ser fundamental.

CONCLUSIONES.

- La ingeniería de software se diferencia de las ciencias de la computación por su enfoque pragmático y sistemático, orientado a construir productos de alta calidad que funcionen bajo restricciones de tiempo y presupuesto en máquinas reales.
- El paso de prácticas empíricas a metodologías estructuradas o ágiles, junto con una fase de inicio que defina claramente el alcance, es determinante para evitar errores de mala calidad que multiplican los costos operativos y generan deuda técnica.
- La aplicación de la norma ISO/IEC 25010 es vital para garantizar que el producto final cumpla con atributos críticos como la mantenibilidad, confiabilidad y eficiencia de desempeño, asegurando que el software satisfaga las necesidades reales del usuario.

RECOMENDACIONES.

- Es fundamental establecer una fase de inicio que determine con precisión el alcance, los límites y los criterios de aceptación. Esto evita que el diseño sea un proceso implícito y previene fallos por requisitos mal definidos.

- Se recomienda adoptar herramientas CASE y metodologías estructuradas o ágiles para automatizar tareas de diseño, prueba y mantenimiento. Esta sistematización es clave para reducir la deuda técnica y optimizar los costos operativos.
- Se sugiere aplicar el modelo ISO/IEC 25010 para evaluar atributos como mantenibilidad, confiabilidad y eficiencia de desempeño. Evaluar la calidad desde las etapas iniciales garantiza que el software sea rentable y trabaje eficazmente en máquinas reales.

BIBLIOGRAFÍA.

Pressman, R. S. (2005). *Software Engineering: A Practitioner's Approach*. 6th Edition. McGraw-Hill.

Sommerville, I. (2011). *Software Engineering*. 9th Edition. Addison-Wesley.

ISO 25010. (s. f.). https://www.iso25000.com/index.php/normas-iso-25000/iso-25010

H. Zhang *et al.*, «Software Performance Engineering for Foundation Model-Powered Software», *ACM Trans. Softw. Eng. Methodol.*, p. 3794844, mar. 2026, doi: 10.1145/3794844.

L. Oliveria De Souza, E. Santana De Almeida, P. A. D. M. Silveira Neto, E. T. Barr, y J. Petke, «Software Product Line Engineering via Software Transplantation», *ACM Trans. Softw. Eng. Methodol.*, vol. 34, n.º 2, pp. 1-27, feb. 2025, doi: 10.1145/3695987.

W. K. G. Assunção, L. Marchezan, L. Arkoh, A. Egyed, y R. Ramler, «Contemporary Software Modernization: Strategies, Driving Forces, and Research Opportunities», *ACM Trans. Softw. Eng. Methodol.*, vol. 34, n.º 5, pp. 1-35, jun. 2025, doi: 10.1145/3708527.

O. Greulich, C. Knieke, B. Rafie, A. Rausch, y M. Kuhrmann, «Software Process as a Service: Towards A Software Process Ecosystem», en *Proceedings of the 2024 International*

Conference on Software and Systems Processes, M\ Germany: ACM, sep. 2024, pp. 77-86. doi: 10.1145/3666015.3666021.

M. Samarah, A. Inuganti, y M. Goyal, «Software Engineering Education: Towards Ethical, Reliable, and Beautiful Software», en *Proceedings of the 1st International Workshop on Designing Software*, Lisbon Portugal: ACM, abr. 2024, pp. 40-45. doi: 10.1145/3643660.3643950.

S. Ibba, G. Baralla, A. Pinna, y R. Tonelli, «Survey: how much the academic startups know and use agile software and lean startup methodologies?», en *Proceedings of the 19th International Conference on Agile Software Development: Companion*, Porto Portugal: ACM, may 2018, pp. 1-3. doi: 10.1145/3234152.3234198.

ANEXOS.

Tabla 1

Integrantes del grupo N°5.

Nombres	**Apellidos**
Joshep Daniel	Chisaguano Reinoso
Cristian David	Suquillo Ayo
Henry Paul	Villacis Arrieta
Melanie Antonella	Zuñiga Narváez

Sangolquí, a 13 de abril de 2026